Water Rich

GitHub Repo: https://github.com/rgotzmann/water-quality-monitor/

Project Board: https://github.com/users/rgotzmann/projects/1

Issues: https://github.com/rgotzmann/water-quality-monitor/issues

Team contract

Romina Gotzmann – full stack engineer with 4 years of experience at private company. Built CI/CD pipeline with Jenkins, company website with custom React theme and plug-ins, experienced with LAMP, RT Linux system administration, C++, Python, and C.

In the terminal semester of M.S. in Data Science.

Roles

Romina: Team, ML, Cloud, and Data Streaming Lead and System Architect.

Coding cadence: weekly sprints to review the code, test and document

Proposal content

The unique geography of the Floridian peninsula means the state has a vast network of intrastate waterways that feed into the ocean. There is also a substantial underground aquifer that provides the state with freshwater. The rivers, streams, canals, retention ponds, lakes, and wetlands are all subject to pollution events caused by agricultural runoff, industrial discharge, and extreme weather events. Traditional water quality monitoring relies on infrequent manual sampling and relatively few sampling locations, so pollution events are often detected days or weeks after they occur or not adequately traced to their point-source. This system aims to provide real-time, continuous monitoring of key water quality parameters, using machine learning to detect anomalies, and trigger alerts within seconds of a sensor reading crossing a threshold. The sensors will be deployed in local canals and retention ponds to provide an initial performance measure. Once proven successful, these low-cost water monitors can be implemented across municipal and state waterways to improve water quality at scale.

User-Facing API Contract & Service Level Objectives (SLO)

Endpoint	Method	Description	SLO Latency	SLO Availability
POST /ingest/reading	Kafka Producer	Ingest sensor event	< 100 ms p99	99.9%
GET /predictions/latest	REST / Kafka Consumer	Latest anomaly score per sensor	< 200 ms p99	99.9%
GET /alerts/active	REST	Current open alerts	< 500 ms p99	99.5%
GET /history/{sensor_id}	REST	Historical readings + scores	< 1 s p99	99.0%
GET /health	REST	System health check	< 50 ms p99	99.99%

Architecture diagram

Data Flow

The system follows a classic event-driven, producer-consumer structure built on Apache Kafka:

```
Sensor / USGS API → Kafka Producer → [readings.raw]  →  Kafka Consumer (ML
Inference) → [predictions.out]  →  Alert Engine → [alerts.active] → Dashboard
```

Components

Layer	Component	Technology	Notes
Ingestion	Sensor Producer	Python + confluent-kafka	Polls USGS API or physical sensor
Streaming	Kafka Broker	Apache Kafka 3.x	Topics: readings.raw, predictions.out, alerts.active
Inference	Stream Consumer	Python + scikit-learn	Isolation Forest anomaly detector

Storage	Time-series DB	InfluxDB / PostgreSQL	Stores all readings + scores
Registry	Model Registry	MLflow / local filesystem	Versioned model artifacts + metadata
Monitoring	Dashboard	Grafana / Streamlit	Live sensor readings + alert feed
Secrets	Config/Secrets	.env + python-dotenv	API keys, broker config

Topics

- **Topic 1:** readings.raw

Raw sensor events. Schema: { sensor_id, timestamp, pH, dissolved_oxygen, turbidity, temperature, conductivity }

- **Topic 2:** predictions.out

ML inference output. Schema: { request_id, sensor_id, timestamp, anomaly_score, is_alert, model_version }

- **Topic 3:** alerts.active

Filtered high-severity alerts only (anomaly_score > threshold). Consumed by dashboard and notification service.

Cloud & Tooling**

All components run locally via Docker for development and deployment. The tooling choices below prioritize zero cost and minimal setup:

Category	Tool / Service	Rationale	Cost Tier
Runtime	Docker + docker-compose	Local dev parity, easy demo	
Kafka	apache/kafka Docker image	Official, no Zookeeper needed (KRaft)	
Model Registry	MLflow (local)	Track experiments, version models	

Object Store	Local filesystem / MinIO	Store model .pkl artifacts	Free
Secrets	python-dotenv (.env file)	Simple, git-ignored	
Dashboard	Streamlit	Fast Python-native UI	
CI/CD	GitHub Actions	Free for public repos, integrates natively	

** Note: The entire stack can run on a single laptop via docker-compose with no cloud account required. If scaling to production, Confluent Cloud (managed by Kafka) and AWS S3 (model artifact storage) would be used.

Data plan

Primary Data Sources

Water quality data is sourced from publicly available government monitoring networks, which provide real historical readings from sensors installed in rivers, lakes, and streams across the United States. The primary source is the USGS Water Quality Portal (waterqualitydata.us), which provides measurements of pH, dissolved oxygen, temperature, conductivity, turbidity, and nitrates.

Source	Format	Use	Replay Strategy
USGS Water Quality Portal	CSV download	Historical baseline for model training	Replay at 100x speed via Kafka producer with time.sleep()
EPA Water Quality Portal	REST API / CSV	Supplement training data, additional parameters	Batch download → replay same as USGS
OpenAQ API	REST API (live)	Live demo events (correlated environmental data)	Live polling every 60s → produce to Kafka
Synthetic generator	Python script	Inject controlled anomalies for demo	Gaussian noise + spike injection script

Event Replay

Since physical sensors are not required for the initial build, historical CSV data from USGS is replayed through a Kafka producer to simulate a real-time stream. The producer reads rows in chronological order and uses time.sleep() to throttle event emission, mimicking real sensor polling intervals. A configurable REPLAY_SPEED multiplier allows the stream to be run at 1x (real-time) or 100x for testing and demo purposes.

- Training split: 80% of historical data used for fitting the Isolation Forest baseline
- Test split: 20% held out for evaluation; known pollution events (if labelled) used for anomaly recall metrics
- Demo mode: Synthetic anomaly injection script overlays-controlled spikes on the live stream for real-time demo

ML-Ops

An Isolation Forest model is used for anomaly detection because it handles multivariate sensor data with no labelled anomaly training examples. It learns the distribution of 'normal' readings and scores new events by how isolated they are from the normal cluster. A configurable contamination parameter (default 0.05) controls the expected anomaly rate.

Area	Detail
Model versioning	Semantic versioning: isolation_forest_v{major}.{minor}.pkl — major bump on retrain with new data, minor on hyperparameter change
Registry metadata fields	model_name, version, trained_at, training_rows, feature_cols, roc_auc, pr_auc, contamination_rate, git_commit, author
CI pipeline	GitHub Actions: lint (flake8) → unit tests (pytest) → integration test (spin up Kafka, produce 100 events, assert predictions flow) → push to registry
Testing strategy	Unit: feature extraction logic. Integration: Kafka producer → consumer round-trip. Model: assert anomaly detection rate within expected range on held-out test set.
Retraining trigger	Manual for v1 (weekly). Future: automated drift detection using population stability index on incoming feature distributions.

CI/CD outline

- Trigger: Pull request to main branch

- Step 1 — Lint: flake8 on all Python files
- Step 2 — Unit tests: pytest tests/unit/ (feature extraction, schema validation)
- Step 3 — Integration test: spin up Kafka via docker-compose, produce 100 synthetic events, assert predictions topic receives 100 messages within 30s
- Step 4 — Model validation: retrain on test dataset, assert PR-AUC >= 0.60
- Step 5 — Registry push: if all pass, log model to MLflow with full metadata

Milestone GANTT

The table shows a 5-week schedule for the remainder of the Spring 2026 semester.

Milestone	Start	End	Status
M1 — GitHub setup, technical proposal	Week 1	Week 1	Completed
M2 — USGS data pipeline + Kafka producer	Week 2	Week 2	To Do
M3 — Evaluation, Pipeline, CI/CD	Week 3	Week 3	To Do
M4 — Docker deploy, retraining, monitoring, experimentation	Week 4	Week 4	To Do
M5 — Fairness, security, feedback loops, final demo	Week 5	Week 5	To Do

Risk Register

The following table captures the top five risks identified for this project with preventive mitigations.

Risk	Mitigation	Contingency
USGS API rate limits or downtime blocks data ingestion	Cache responses locally; implement retry with backoff	Use CSV historical data
Kafka broker crashes	Use docker restart policy; test locally before demo	Pre-record a screen capture of working system
Model detects too few anomalies	Lower contamination threshold; tune decision threshold	Use synthetic anomaly injection script to force alerts

Data schema changes between USGS API versions	Use specific API version; store feature_cols.json	Manual column mapping in producer
Consumer lag grows faster than producer (throughput)	Increase Kafka partitions; add consumer group members	Throttle producer with time.sleep()